

Assignment - 6 (30-04-2026)

Using Bookstore Swagger API, perform the following requests:

GET, POST, PUT, PATCH, DELETE

Write test scripts for the same.

Save the responses and capture the test results

POST - (User creation)

Prerequisites: Authentication

Account Creation

Before we can get a token, we need a user.

1. Open postman and enter the url under **POST**: <https://bookstore.toolsqa.com/Account/v1/User>
2. In the Body, enter a username and password

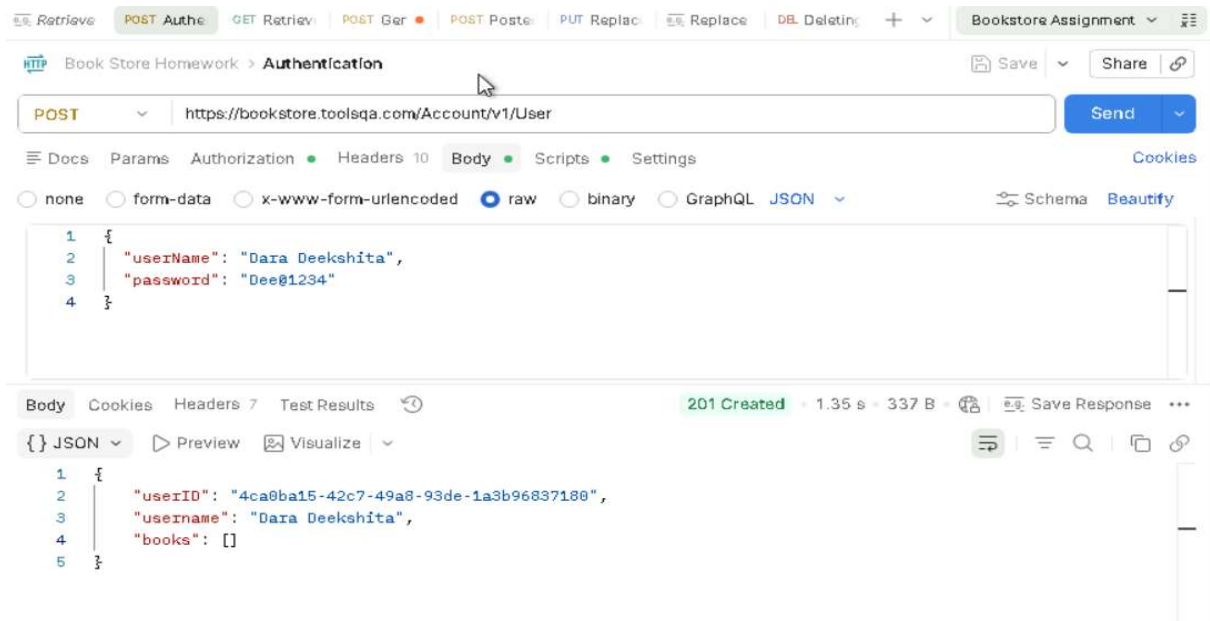
```
{  
  
  "userName": "Dara Deekshita",  
  
  "password": "Dee@1234"  
}
```

3. In test tab write the below lines

```
const response = pm.response.json();
```

```
pm.environment.set("token", response.token);
```

Now the user is created and saved as a token. This token can be used for authentication.



1. GET - (Get all Books)

URL: https://bookstore.toolsqa.com/BookStore/v1/Books

Body: None

Script:

```
pm.test("Status code is 200", function () {

    pm.response.to.have.status(200);

});

pm.test("Response is valid JSON", function () {

    try {

        const jsonData = pm.response.json();
```

```

    pm.expect(jsonData).to.be.an('object');

    } catch (e) {

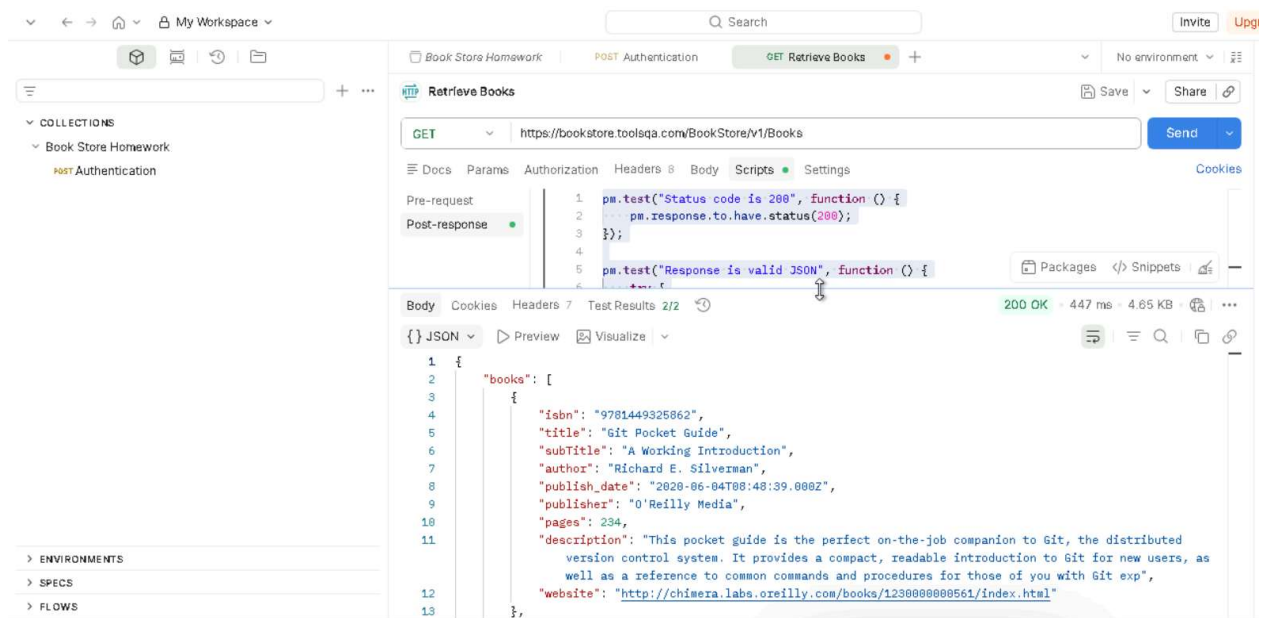
    pm.expect.fail("Expected JSON but received: " +
pm.response.text().substring(0, 20));

    }

});

```

Results:



The screenshot shows the Postman interface with a collection named 'Book Store Homework' and a folder 'POST Authentication'. A request named 'Retrieve Books' is selected, showing a GET method to the URL 'https://bookstore.toolsqa.com/BookStore/v1/Books'. The response is a 200 OK status with a response time of 447 ms and a body size of 4.65 KB. The response body is a JSON array of book objects. The first book object is expanded, showing the following details:

```

{
  "books": [
    {
      "isbn": "9781449325862",
      "title": "Git Pocket Guide",
      "subTitle": "A Working Introduction",
      "author": "Richard E. Silverman",
      "publish_date": "2020-06-04T08:48:39.000Z",
      "publisher": "O'Reilly Media",
      "pages": 234,
      "description": "This pocket guide is the perfect on-the-job companion to Git, the distributed version control system. It provides a compact, readable introduction to Git for new users, as well as a reference to common commands and procedures for those of you with Git exp",
      "website": "http://chimera.labs.oreilly.com/books/12300000000561/index.html"
    }
  ]
}

```

Generate Token

URL: <https://bookstore.toolsqa.com/Account/v1/GenerateToken>

Body:

```
{
```

```
"userName": "Dara Deekshita",  
  
"password": "Dee@1234"  
  
}
```

Script:

```
// Parse the JSON response  
  
const response = pm.response.json();  
  
// Save the token to an environment variable named "token"  
  
pm.environment.set("token", response.token);  
  
// Optional: Test to confirm it saved correctly  
  
pm.test("Token was captured and saved", function () {  
  
    pm.expect(pm.environment.get("token")).to.not.be.undefined;  
  
});
```

Results:

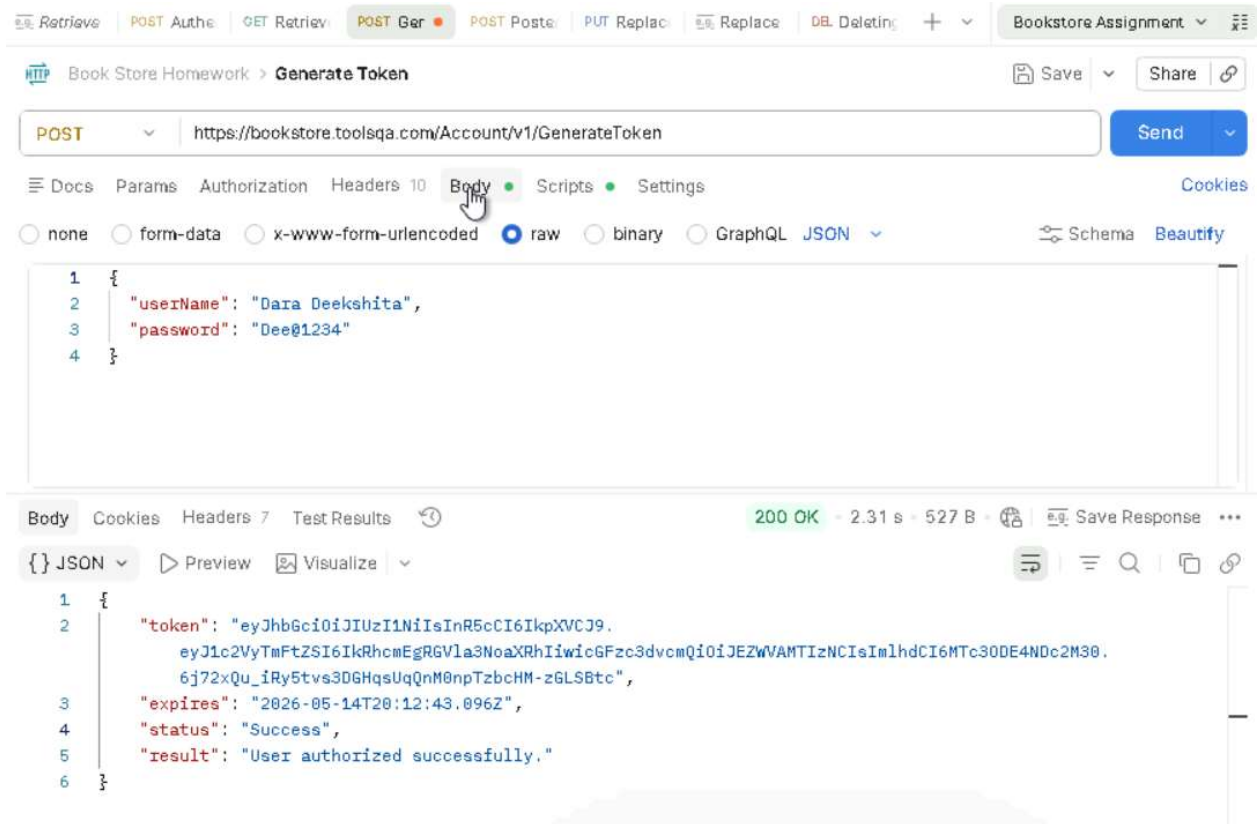
Use this for every POST,GET,PATCH commands

Open your next request (e.g., POST /BookStore/v1/Books).

Go to the Authorization tab.

Select Type: Bearer Token.

In the Token field, type: {{token}}.



2. POST (Add a Book to Collection)

URL: <https://bookstore.toolsqa.com/BookStore/v1/Books>

Body:

```
{
  "userId": "9b5f1234-5678-4abc-8def-1234567890ab",
  "collectionOfIsbns": [
    {
      "isbn": "9781449325862"
```

```

    }
  ]
}

```

Script:

```

pm.test("Status code is 201 (Created)", function () {

pm.response.to.have.status(201); });

```

Results:

The screenshot displays the Postman interface for a REST client. The top bar shows a search bar and buttons for 'Invite' and 'Upg'. The main area is divided into several tabs: 'Book Store', 'POST Auth', 'GET Retrie', 'POST Ger', 'POST http', 'Authenti', 'Retrieve', 'Booksto', and 'Bookstore Assignment'. The selected tab is 'POST http', which shows the URL 'https://bookstore.toolsqa.com/BookStore/v1/Books'. The request method is 'POST', and the request body is set to 'raw' (JSON). The body content is a JSON object with a 'userId' and a 'collectionOfIsbns' array containing one object with an 'isbn' value. The response status is '201 Created', and the response body is a JSON object with a 'books' array containing one object with an 'isbn' value. The response is displayed in the 'Body' tab, which also shows 'JSON' format, 'Preview', and 'Visualize' options.

```

1 {
2   "userId": "4ca0ba15-42c7-49a8-93de-1a3b96837180",
3   "collectionOfIsbns": [
4     {
5       "isbn": "9781449337711"
6     }
7   ]
8 }

```

```

1 {
2   "books": [
3     {
4       "isbn": "9781449337711"
5     }
6   ]
7 }

```

3. PUT - Replace the book ISBN

URL: <https://bookstore.toolsqa.com/BookStore/v1/Books/9781449337711>

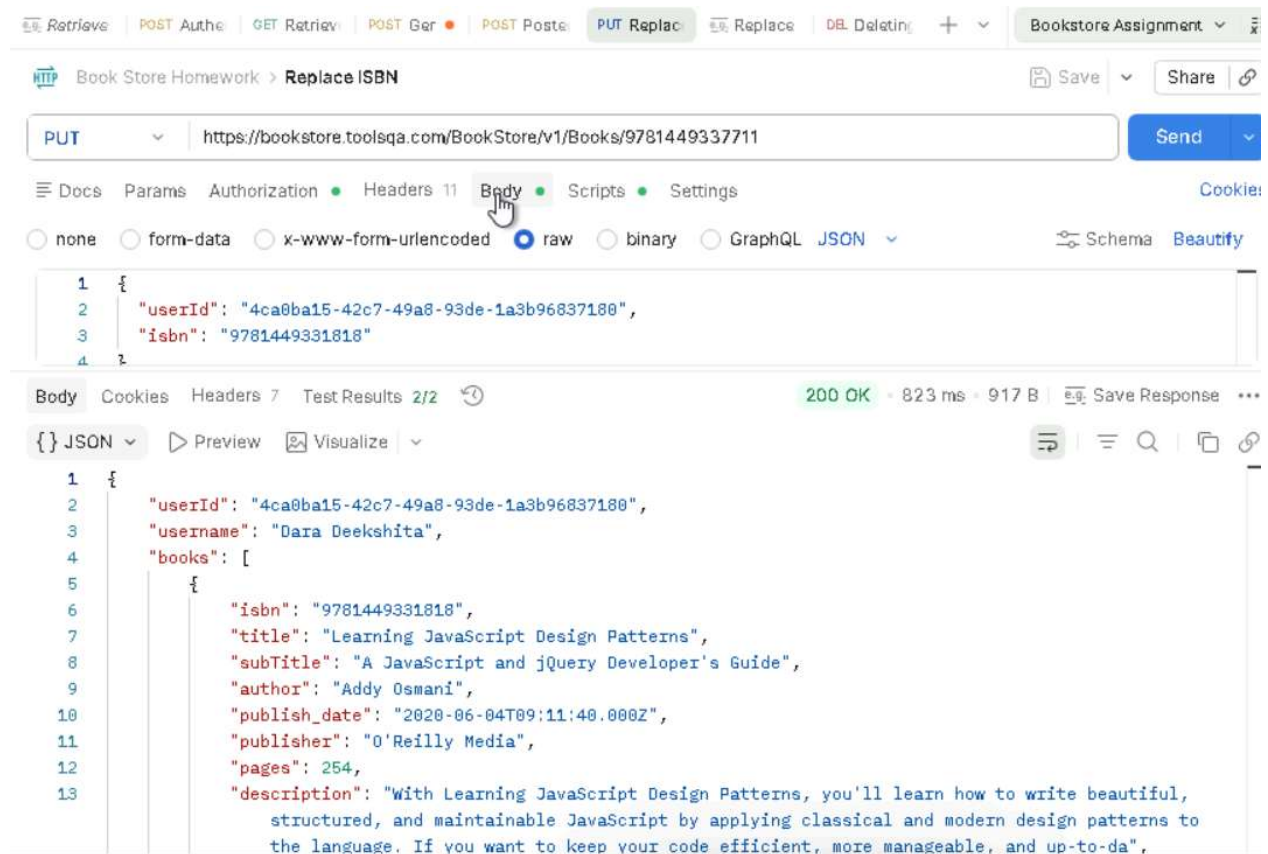
Body:

```
{  
  
  "userId": "4ca0ba15-42c7-49a8-93de-1a3b96837180",  
  
  "isbn": "9781449331818"  
}
```

Script:

```
pm.test("Status code is 200", function () {  
  
  pm.response.to.have.status(200);  
  
});  
  
pm.test("Book replaced successfully", function () {  
  
  const jsonData = pm.response.json();  
  
  pm.expect(jsonData.username).to.not.be.null;  
  
  pm.expect(jsonData.books).to.be.an('array');  
  
});
```

Result:



There is no patch endpoint. Here it is similar to PUT

4. DELETE - delete a book

URL: https://bookstore.toolsqa.com/BookStore/v1/Books

Body:

```
{  
  
  "isbn": "9781449331818",  
  
  "userId": "4ca0ba15-42c7-49a8-93de-1a3b96837180"  
}
```

Script:

```
pm.test("Status code is 204", function () {

    // 204 No Content is the standard success code for DELETE

    pm.expect(pm.response.code).to.be.oneOf([204, 200]);

});

pm.test("Book deleted successfully", function () {

    if(pm.response.code === 204) {

        pm.expect(true).to.be.true;

    } else {

        const jsonData = pm.response.json();

        pm.expect(jsonData.message).to.not.include("Error");

    }

});
```

Result:

PASSED Book deleted successfully

Retrieval POST Auth GET Retrieval POST Get POST Post PUT Replace Replace DEL https Bookstore Assignment

https://bookstore.toolsqa.com/BookStore/v1/Books Save Share

DELETE https://bookstore.toolsqa.com/BookStore/v1/Books Send

Docs Params Authorization Headers 11 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "isbn": "9781449331818",
3   "userId": "4ca0ba15-42c7-49a8-93de-1a3b96837180"
4 }
```

Body Cookies Headers 7 Test Results 1/2 401 Unauthorized 135 ms 302 B

Testing your API endpoints one by one? Run your entire collection and test end-to-end workflows effortlessly. [Learn more](#) Run Collection